

的判空接口`empty()`，以及区间删除接口`remove(lo, hi)`、区间查找接口`find(e, lo, hi)`等。它们多为上述基本接口的扩展或变型，可使代码更为简洁易读，并提高计算效率。

这里还提供了`sort()`接口，以将向量转化为有序向量。为此可有多种排序算法供选用，本章及后续章节，将陆续介绍它们的原理、实现并分析其效率。排序之后，向量的很多操作都可更加高效地完成，其中最基本和最常用的莫过于查找。因此，这里还针对有序向量提供了`search()`接口，并将详细介绍若干种相关的算法。为便于对`sort()`算法的测试，这里还设有一个`unsort()`接口，以将向量随机置乱。在讨论这些接口之前，我们首先介绍基本接口的实现。

§ 2.3 构造与析构

由代码2.1可见，向量结构在内部维护一个元素类型为`T`的私有数组`_elem[]`：其容量由私有变量`_capacity`指示；有效元素的数量（即向量当前的实际规模），则由`_size`指示。此外还进一步地约定，在向量元素的秩、数组单元的逻辑编号以及物理地址之间，具有如下对应关系：

向量中秩为`r`的元素，对应于内部数组中的`_elem[r]`，其物理地址为`_elem + r`

因此，向量对象的构造与析构，将主要围绕这些私有变量和数据区的初始化与销毁展开。

2.3.1 默认构造方法

与所有的对象一样，向量在使用之前也需首先被系统创建——借助构造函数（`constructor`）做初始化（`initialization`）。由代码2.1可见，这里为向量重载了多个构造函数。

其中默认的构造方法是，首先根据创建者指定的初始容量，向系统申请空间，以创建内部私有数组`_elem[]`；若容量未明确指定，则使用默认值`DEFAULT_CAPACITY`。接下来，鉴于初生的向量尚不包含任何元素，故将指示规模的变量`_size`初始化为0。

整个过程顺序进行，没有任何迭代，故若忽略用于分配数组空间的时间，共需常数时间。

2.3.2 基于复制的构造方法

向量的另一典型创建方式，是以某个已有的向量或数组为蓝本，进行（局部或整体的）克隆。代码2.1中虽为此功能重载了多个接口，但无论是已封装的向量或未封装的数组，无论是整体还是区间，在入口参数合法的前提下，都可归于如代码2.2所示的统一的`copyFrom()`方法：

```
1 template <typename T> //元素类型
2 void Vector<T>::copyFrom(T const* A, Rank lo, Rank hi) { //以数组区间A[lo, hi)为蓝本复制向量
3     _elem = new T[_capacity = 2 * (hi - lo)]; _size = 0; //分配空间，规模清零
4     while (lo < hi) //A[lo, hi)内的元素逐一
5         _elem[_size++] = A[lo++]; //复制至_elem[0, hi - lo)
6 }
```

代码2.2 基于复制的向量构造器

`copyFrom()`首先根据待复制区间的边界，换算出新向量的初始规模；再以双倍的容量，为内部数组`_elem[]`申请空间。最后通过一趟迭代，完成区间`A[lo, hi)`内各元素的顺次复制。

若忽略开辟新空间所需的时间，运行时间应正比于区间宽度，即 $O(hi - lo) = O_size)$ 。

需强调的是，由于向量内部含有动态分配的空间，默认的运算符“=”不足以支持向量之间的直接赋值。例如，6.3节将以二维向量形式实现图邻接表，其主向量中的每一元素本身都是一维向量，故通过默认赋值运算符，并不能复制向量内部的数据区。

为适应此类赋值操作的需求，可如代码2.3所示，重载向量的赋值运算符。

```
1 template <typename T> Vector<T>& Vector<T>::operator=(Vector<T> const& V ) { //重载赋值操作符
2     if (_elem) delete [] _elem; //释放原有内容
3     copyFrom(V._elem, 0, V.size()); //整体复制
4     return *this; //返回当前对象的引用，以便链式赋值
5 }
```

代码2.3 重载向量赋值操作符

2.3.3 析构方法

与所有对象一样，不再需要的向量，应借助析构函数（**destructor**）及时清理（**cleanup**），以释放其占用的系统资源。与构造函数不同，同一对象只能有一个析构函数，不得重载。

向量对象的析构过程，如代码2.1中的方法~Vector()所示：只需释放用于存放元素的内部数组_elem[]，将其占用的空间交还操作系统。_capacity和_size之类的内部变量无需做任何处理，它们将作为向量对象自身的一部分被系统回收，此后既无需也无法被引用。

若不计系统用于空间回收的时间，整个析构过程只需 $O(1)$ 时间。

同样地，向量中的元素可能不是程序语言直接支持的基本类型。比如，可能是指向动态分配对象的指针或引用，故在向量析构之前应该提前释放对应的空间。出于简化的考虑，这里约定并遵照“谁申请谁释放”的原则。究竟应释放掉向量各元素所指的對象，还是需要保留这些对象，以便通过其它指针继续引用它们，应由上层调用者负责确定。

§ 2.4 动态空间管理

2.4.1 静态空间管理

内部数组所占物理空间的容量，若在向量的生命期内不允许调整，则称作静态空间管理策略。很遗憾，该策略的空间效率难以保证。一方面，既然容量固定，总有可能在此后的某一时刻，无法加入更多的新元素——即导致所谓的上溢（**overflow**）。例如，若使用向量来记录网络访问日志，则由于插入操作远多于删除操作，必然频繁溢出。注意，造成此类溢出的原因，并非系统不能提供更多的空间。另一方面反过来，即便愿意为降低这种风险而预留出部分空间，也很难在程序执行之前，明确界定一个合理的预留量。以上述copyFrom()方法为例，即便将容量取作初始规模的两倍，也只能保证在此后足够长的一段时间内（而并非永远）不致溢出。

向量实际规模与其内部数组容量的比值（即_size/_capacity），亦称作装填因子（**load factor**），它是衡量空间利用率的重要指标。从这一角度，上述难题可归纳为：

如何才能保证向量的装填因子既不致于超过1，也不致于太接近于0？

为此，需要改用动态空间管理策略。其中一种有效的方法，即使用所谓的可扩充向量。